



42-MAVZU · 42 TADAN · V BOB. DRF VA DEPLOY

Yakuniy nazorat ishi — IV–V boblar (Django, DRF, Deploy) va yakuniy loyiha

Django, Django REST Framework va deploy bo'yicha nazorat: 25 ta yopiq test va 5 ta ochiq topshiriq (jami 100 ball); yakuniy loyiha talablari, g'oyalar, baholash mezonlari va taqdimot rejasi.

1 soat · nazorat ishi

Nazorat ishi tuzilishi

- **1-qism:** 25 ta yopiq test × 3 ball = **75 ball** (Python+PostgreSQL — 3, Django — 14, DRF — 5, Deploy — 3).
- **2-qism:** 5 ta ochiq savol va amaliy topshiriq × 5 ball = **25 ball**.
- **Jami: 100 ball.** Vaqt: 80 daqiqa.
- Sahifa oxirida — kursning **yakuniy loyihasi** talablari (31-loyiha).

Takrorlash rejasi

Mavzu	Bilishingiz kerak
32. Python + PostgreSQL	venv, psycopg, connect/execute/fetch, commit, parametrli so'rovlar va SQL injection
33. Django'ga kirish	MVT, project/app, startproject/startapp, INSTALLED_APPS, runserver, settings, DATABASES
34. URL va View	request/response, path, include, converterlar, name/reverse, Http404, redirect, CBV

35. Template va static	render, {{ }}, {% %}, filtrlar, extends/block/include, {% url %}, {% static %}
36. Model va ORM	maydon turlari, ForeignKey, makemigrations/migrate, objects.all/filter/get/create, lookups
37. Admin va Forms	admin.register, ModelAdmin, Form/ModelForm, is_valid, csrf_token, login_required, UserCreationForm
38. CRUD	5 sahifa, instance=, ruxsatlar, POST bilan o'chirish, Paginator, generic CBV'lar
39–40. DRF	REST, JSON, metodlar va status kodlar, Serializer, @api_view, generics, ViewSet, Router, permissions, CORS
41. Deploy	DEBUG=False, SECRET_KEY, ALLOWED_HOSTS, gunicorn, whitenoise, collectstatic, muhit o'zgaruvchilari

1-qism: Yopiq testlar (25 × 3 = 75 ball)

? 1. SQL injection'dan himoyalangan so'rov qaysi?

- A. `cur.execute(f"SELECT * FROM t WHERE ism='{ism}'")`
- B. `cur.execute("SELECT * FROM t WHERE ism=%s", (ism,))`
- C. `cur.execute("SELECT * FROM t WHERE ism='" + ism + "'")`
- D. Hammasi xavfsiz

? 2. psycopg'da INSERT dan keyin o'zgarishni saqlash?

- A. `cur.save()`
- B. `cur.fetchall()`
- C. `conn.close()`
- D. `conn.commit()` (yoki `with` blokining muvaffaqiyatli tugashi)

? 3. Loyiha kutubxonalarini alohida saqlash uchun nima yaratiladi?

- A. Virtual muhit (venv)
- B. Yangi baza
- C. Git branch
- D. Docker

? 4. Django arxitekturasidagi "T" nima?

- A. Table
- B. Test
- C. Template
- D. Token

? 5. Yangi ilova yaratilgandan keyin uni qayerga qo'shish kerak?

- A. `urls.py`
- B. `settings.py` → `INSTALLED_APPS`
- C. `models.py`
- D. `manage.py`

? 6. `path("kitob/<int:pk>/", views.kitob)` — view qanday e'lon qilinadi?

- A. `def kitob(request):`
- B. `def kitob(pk):`
- C. `def kitob(request, id):`
- D. `def kitob(request, pk):`

? 7. Ilova URL'larini asosiy faylga ulash?

- A. `path("", include("blog.urls"))`
- B. `import blog.urls`
- C. `urls.append(blog)`
- D. `path("", blog)`

? 8. Shablonda o'zgaruvchi chiqarish?

- A. `{% ism %}`
- B. `[[ ism ]]`
- C. `{{ ism }}`
- D. `${ ism }`

? 9. Umumiy skeletdan meros olish tegi?

- A. `{% include %}`
- B. `{% extends %}`
- C. `{% block %}`
- D. `{% load %}`

? 10. Shablonda CSS ulash uchun avval nima yoziladi?

- A. `{% css %}`
- B. `{% import static %}`
- C. `{% url 'static' %}`
- D. `{% load static %}`

? 11. Model o'zgargandan keyin bajariladigan buyruqlar ketma-ketligi?

- A. `makemigrations` → `migrate`
- B. `migrate` → `makemigrations`
- C. `runserver` → `migrate`
- D. `collectstatic`

? 12. "Har bir kitobning bitta muallifi, muallifning ko'p kitobi" — Kitob modelida qaysi maydon?

- A. `ManyToManyField(Muallif)`
- B. `CharField`
- C. `ForeignKey(Muallif, on_delete=...)`
- D. `OneToOneField`

? 13. Narxi 50 000 dan kichik kitoblar?

- A. `Kitob.objects.get(narx<50000)`
- B. `Kitob.objects.filter(narx__lt=50000)`
- C. `Kitob.objects.all(narx=50000)`
- D. `Kitob.filter("narx < 50000")`

? 14. Obyekt topilmasa avtomatik 404 qaytaruvchi funksiya?

- A. `Model.objects.get()`
- B. `Http404()`
- C. `find()`
- D. `get_object_or_404()`

? 15. Admin ro'yxatidagi ustunlarni belgilovchi sozlama?

- A. `list_display`
- B. `fields`
- C. `columns`
- D. `search_fields`

? 16. POST formada majburiy xavfsizlik tegi?

- A. `{% secure %}`
- B. `{% load csrf %}`
- C. `{% csrf_token %}`
- D. `{% token %}`

? 17. Mavjud obyektни ModelForm bilan tahrirlash?

- A. `Form(request.POST, id=5)`
- B. `Form(request.POST, instance=obj)`
- C. `Form.update(obj)`
- D. `obj.form()`

? 18. Faqat tizimga kirganlar uchun ochiq view (funksiya)?

- A. `@admin_required`
- B. `@csrf_exempt`
- C. `@api_view`
- D. `@login_required`

? 19. REST API'da resursni o'chirish metodi va javob kodi?

- A. GET, 200
- B. POST, 201
- C. DELETE, 204
- D. PUT, 404

? 20. Model ↔ JSON aylantiruvchi DRF klassi?

- A. `ModelSerializer`
- B. `ModelForm`
- C. `ModelAdmin`
- D. `JsonResponse`

? 21. Bir nechta obyektни serializatsiya qilish?

- A. `Serializer(qs, all=True)`
- B. `Serializer(qs, many=True)`
- C. `Serializer.list(qs)`
- D. `Serializer(qs)`

? 22. To'liq CRUD API'ni bitta klassda beruvchi DRF klassi va URL'larni avtomatik yaratuvchi vosita?

- A. `APIView` + `path`
- B. `Serializer` + `include`
- C. `ListView` + `Paginator`
- D. `ModelViewSet` + `DefaultRouter`

? 23. Boshqa domendagi JS sayt API'ga murojaat qila olishi uchun Django'da nima sozlanadi?

- A. CSRF
- B. ALLOWED_HOSTS
- C. CORS (`django-cors-headers`)
- D. WhiteNoise

? 24. Production'da Django ilovasini ishga tushiruvchi server dasturi?

- A. `gunicorn`
- B. `runserver`
- C. `pip`
- D. `psql`

? 25. Internetdagi saytda qaysi sozlama noto'g'ri?

- A. `DEBUG = False`
- B. `DEBUG = True`
- C. `ALLOWED_HOSTS = ["sayt.onrender.com"]`
- D. `SECRET_KEY` muhit o'zgaruvchisida

2-qism: Ochiq savollar va amaliy topshiriqlar (5 × 5 = 25 ball)

1-topshiriq (5 ball). Nazariy savol

Foydalanuvchi brauzerda `/maqola/5/` ni ochganda Django ichida nima sodir bo'lishini MVT tushunchalari bilan bosqichma-bosqich tushuntiring.

▼ Namuna javob va baholash

(1) So'rov Django'ga keladi; (2) `urls.py` yo'llarni tekshirib, `maqola/<int:id>/` ga mosini topadi va `id=5` bilan view'ni chaqiradi; (3) View Model orqali (ORM) bazadan 5-maqolani so'raydi — `get_object_or_404(Maqola, id=5)`; topilmasa 404; (4) View maqolani kontekst sifatida Template'ga beradi — `render(...)`; (5) Shablon ma'lumot bilan to'ldirilib, tayyor HTML javob brauzerga qaytariladi.

Baholash: URL — 1; View — 1; Model/ORM/baza — 1; Template/render — 1; to'g'ri ketma-ketlik va 404 holati — 1.

2-topshiriq (5 ball). Model va admin

`Kitob` modelini yozing: nomi (200 belgi), muallif (100 belgi), janr (tanlov: roman, she'riyat, ilmiy), yil, narx (pul), mavjud (standart — ha), qo'shilgan sana (avtomatik). `__str__` nomini qaytarsin.

Admin'da ro'yxatda nomi, muallif, janr, mavjud ko'rinsin; janr va mavjudlik bo'yicha filtr, nom va muallif bo'yicha qidiruv.

▼ Namuna javob

Python

```
from django.db import models
from django.contrib import admin

class Kitob(models.Model):
    JANRLAR = [("roman", "Roman"), ("sheriyat", "She'riyat"),
               ("ilmiy", "Ilmiy")]
    nomi = models.CharField(max_length=200)
    muallif = models.CharField(max_length=100)
    janr = models.CharField(max_length=20, choices=JANRLAR)
    yil = models.PositiveIntegerField()
    narx = models.DecimalField(max_digits=10, decimal_places=2)
    mavjud = models.BooleanField(default=True)
    qoshilgan = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return self.nomi

@admin.register(Kitob)
class KitobAdmin(admin.ModelAdmin):
    list_display = ["nomi", "muallif", "janr", "mavjud"]
    list_filter = ["janr", "mavjud"]
    search_fields = ["nomi", "muallif"]
```

Baholash: maydon turlari to'g'ri — 2; choices va default/auto_now_add — 1; `__str__` — 1; admin sozlamalari — 1.

3-topshiriq (5 ball). View va shablon

Mavjud kitoblar ro'yxatini ko'rsatuvchi view va shablon yozing: `?q=` parametri bo'lsa, nomi bo'yicha qidirsin; ro'yxat bo'sh bo'lsa "Kitob topilmadi" chiqsin; har bir kitob nomi batafsil

sahifaga havola bo'lsin.

▼ Namuna javob

Python

```
def kitoblar(request):
    q = request.GET.get("q", "").strip()
    qs = Kitob.objects.filter(mavjud=True)
    if q:
        qs = qs.filter(nomi__icontains=q)
    return render(request, "kutubxona/kitoblar.html", {"kitoblar": qs,
"q": q})
```

Django shablon

```
{% extends "kutubxona/base.html" %}
{% block content %}
    <form><input name="q" value="{{ q }}" placeholder="Qidirish">
<button>Izlash</button></form>
    <ul>
        {% for k in kitoblar %}
            <li><a href="{% url 'kutubxona:kitob' k.id %}">{{ k.nomi }}</a>
– {{ k.muallif }}</li>
        {% empty %}
            <li>Kitob topilmadi</li>
        {% endfor %}
    </ul>
{% endblock %}
```

Baholash: filter va icontains — 1; GET parametr — 1; render va kontekst — 1; for/empty — 1;

{% url %} bilan havola — 1.

4-topshiriq (5 ball). DRF API

`Kitob` uchun to'liq CRUD API yarating (`/api/kitoblar/`): o'qish — hammaga, o'zgartirish — faqat tizimga kirganlarga; narx musbat bo'lishi tekshirilsin; nom bo'yicha qidiruv.

▼ Namuna javob

Python

```
from rest_framework import serializers, viewsets, permissions, filters
from rest_framework.routers import DefaultRouter

class KitobSerializer(serializers.ModelSerializer):
    class Meta:
        model = Kitob
        fields = "__all__"

    def validate_narx(self, v):
        if v <= 0:
            raise serializers.ValidationError("Narx musbat bo'lishi kerak")
        return v

class KitobViewSet(viewsets.ModelViewSet):
    queryset = Kitob.objects.all()
    serializer_class = KitobSerializer
    permission_classes = [permissions.IsAuthenticatedOrReadOnly]
    filter_backends = [filters.SearchFilter]
    search_fields = ["nomi"]

router = DefaultRouter()
router.register("kitoblar", KitobViewSet)
# urls.py: path("api/", include(router.urls))
```

Baholash: serializer — 1; validatsiya — 1; ViewSet — 1; ruxsat — 1; router va qidiruv — 1.

5-topshiriq (5 ball). Deploy

Django loyihasini Render'ga joylash uchun bajariladigan asosiy qadamlarni va production sozlamalarini sanab, har birini qisqa tushuntiring.

▼ Namuna javob

1. `gunicorn`, `whitenoise`, `django-database-url` o'rnatish va `requirements.txt` ni yangilash.
2. `settings.py`: `DEBUG=False`, `SECRET_KEY` va baza — muhit o'zgaruvchilaridan, `ALLOWED_HOSTS` va `CSRF_TRUSTED_ORIGINS` — domen, `WhiteNoise` middleware, `STATIC_ROOT`.
3. `build.sh`: `pip install` → `collectstatic` → `migrate`. Kod GitHub'ga push.
4. Render'da PostgreSQL va Web Service; Start: `gunicorn config.wsgi:application`; Environment Variables.
5. Deploy'dan keyin: `superuser` yaratish, tekshiruv ro'yxati, Logs orqali xatolarni topish.

Baholash: har bir to'g'ri tushuntirilgan bosqich — 1 ball.

Baholash shkalasi

Ball	Natija
86–100	A'lo — kurs dasturini to'liq o'zlashtirgansiz
71–85	Yaxshi
60–70	Qoniqarli
0–59	Qoniqarsiz — mentor bilan takrorlash tavsiya etiladi

Yakuniy loyiha (31-loyiha)

Kurs davomida o'rganilgan **barcha** bilimlarni bitta loyihada namoyish eting. Loyiha individual yoki 2–3 kishilik jamoada (13-mavzu) bajariladi.

Majburiy talablar

Qism	Talab	Mavzular
Front-end	Semantik HTML, o'z CSS dizayni, responsiv (telefon + kompyuter), kamida bitta JS interaktivlik	2–12, 15–20
Ma'lumotlar bazasi	PostgreSQL; kamida 3 ta model, kamida bitta <code>ForeignKey</code>	26–30, 36
Back-end	Django: CRUD (5 sahifa), shablon merosi, formalar va validatsiya, admin panel	33–38
Foydalanuvchilar	Ro'yxatdan o'tish / kirish; kamida bitta "faqat muallif" ruxsati	37–38
API	Kamida bitta resurs uchun DRF API (GET + POST)	39–40
Algoritmik qism	Shart va sikllarga asoslangan mantiq: statistika, reyting, hisob-kitob, filtrlash	16–18, 21–23, 30
Git va deploy	GitHub'da kamida 15 ta mazmunli commit, README; internetga joylangan	11, 13, 41

Loyiha g'oyalari

- 📖 **Maktab kutubxonasi:** kitoblar katalogi, ijaraga olish/qaytarish, kechikish jarimasi, eng ko'p o'qilgan kitoblar reytingi.
- 📝 **Onlayn test tizimi:** o'qituvchi test yaratadi, o'quvchilar yechadi, natijalar va reyting jadvali.
- 🏠 **Sinf jurnali:** o'quvchilar, fanlar, baholar, davomat; o'rtacha baholar va hisobotlar.
- 🛒 **Maktab do'koni / yarmarka:** mahsulotlar, savat, buyurtmalar, ombor hisobi.
- 🎯 **To'garaklar portali:** to'garaklar, joylar soni, ro'yxatdan o'tish, jadval.
- 💡 **O'z g'oyangiz** — mentor bilan kelishgan holda.

Bajarish rejasi

- 1 G'oya va rejalashtirish** (1-hafta): funksiyalar ro'yxati, ER-diagramma (26-mavzu), Figma'da 2–3 sahifa maketi (10-mavzu), GitHub repozitoriya va Kanban doskasi.
- 2 Back-end** (2-hafta): modellar, migratsiyalar, admin, CRUD view'lar va formalar.

- 3 **Front-end** (3-hafta): shablonlar, dizayn, responsivlik, JS interaktivlik.
- 4 **API, sinov va deploy** (4-hafta): DRF endpoint, sinov ssenariylari va testlar, Render'ga joylash, README.
- 5 **Taqdimot.**

Baholash mezonlari (100 ball)

Mezon	Ball
Funksionallik: barcha majburiy talablar ishlaydi, xatolarsiz	30
Kod sifati: tushunarli nomlar, tartibli tuzilma, takrorlanmaslik, xavfsizlik (CSRF, ruxsatlar, parametrli so'rovlar)	20
Dizayn va foydalanish qulayligi: chiroyli, responsiv, tushunarli	15
Ma'lumotlar bazasi tuzilmasi: to'g'ri modellar va bog'lanishlar	10
Git, README va deploy: commit tarixi, hujjat, ishlaydigan havola	10
Taqdimot: maqsadni, ishlash prinsipini va texnologiyalarni tushuntira olish, savollarga javob	15

Taqdimot rejasi (5–7 daqiqa)

1. **Muammo va g'oya** — loyiha kimga va nima uchun kerak? (30 soniya)
2. **Jonli namoyish** — internetdagi sayt: asosiy ssenariylar (3 daqiqa).
3. **Texnik tuzilma** — modellar diagrammasi, qaysi texnologiyalar qayerda ishlatilgan (1–2 daqiqa).
4. **Qiyinchiliklar va xulosalar** — eng qiyin muammo va uni qanday hal qildingiz; nimani yaxshilagan bo'lardingiz (1 daqiqa).
5. **Savollarga javob.**

Loyiha havolasi o'quv dasturiga ko'ra GitHub repozitoriyasi yoki mentor tasdiqlagan boshqa platforma (Google Drive, Telegram kanal/guruhdagi video-taqdimot) orqali topshiriladi.

Tabriklaymiz — kurs yakunlandi!

Siz 66 soat davomida veb-sahifaning birinchi tegidan internetda ishlaydigan, ma'lumotlar bazasi va API'ga ega full-stack ilovagacha bo'lgan yo'lni bosib o'tdingiz: **HTML → CSS → JavaScript → Python → PostgreSQL → Django → DRF → Deploy**.

Keyingi qadamlar:

- Ushbu saytdagi "Full-stack qo'llanma" kursida JavaScript backend (Node.js, Express) va React bilan tanishing.
- **Telegram botlar** (Python + Aiogram) — Python bilimlaringizni yangi yo'nalishda qo'llang.
- Loyihalar qilishda davom eting: har bir yangi loyiha — portfoliingizga yangi qadam.
- Olimpiadalar, xakatonlar va IT-tanlovlarda qatnashing.

Omad! 

☐ Darsni o'qib bo'ldim

✓ Test javoblari

1. 1. SQL injection'dan himoyalangan so'rov qaysi? — **B) `cur.execute("SELECT * FROM t WHERE ism=%s", (ism,))`**
2. 2. psycopg2'da INSERT dan keyin o'zgarishni saqlash? — **D) `conn.commit()` (yoki with blokining muvaffaqiyatli tugashi)**
3. 3. Loyiha kutubxonalarini alohida saqlash uchun nima yaratiladi? — **A) Virtual muhit (venv)**
4. 4. Django arxitekturasidagi "T" nima? — **C) Template**
5. 5. Yangi ilova yaratilgandan keyin uni qayerga qo'shish kerak? — **B) `settings.py` → `INSTALLED_APPS`**
6. 6. `path("kitob/<int:pk>/", views.kitob)` — view qanday e'lon qilinadi? — **D) `def kitob(request, pk):`**
7. 7. Ilova URL'larini asosiy faylga ulash? — **A) `path("", include("blog.urls"))`**
8. 8. Shablonda o'zgaruvchi chiqarish? — **C) `{{ ism }}`**
9. 9. Umumiy skeletdan meros olish tegi? — **B) `{% extends %}`**
10. 10. Shablonda CSS ulash uchun avval nima yoziladi? — **D) `{% load static %}`**
11. 11. Model o'zgargandan keyin bajariladigan buyruqlar ketma-ketligi? — **A) `makemigrations` → `migrate`**
12. 12. "Har bir kitobning bitta muallifi, muallifning ko'p kitobi" — Kitob modelida qaysi maydon? — **C) `ForeignKey(Muallif, on_delete=...)`**
13. 13. Narxi 50 000 dan kichik kitoblar? — **B) `Kitob.objects.filter(narx__lt=50000)`**
14. 14. Obyekt topilmasa avtomatik 404 qaytaruvchi funksiya? — **D) `get_object_or_404()`**
15. 15. Admin ro'yxatidagi ustunlarni belgilovchi sozlama? — **A) `list_display`**
16. 16. POST formada majburiy xavfsizlik tegi? — **C) `{% csrf_token %}`**
17. 17. Mavjud obyektni ModelForm bilan tahrirlash? — **B) `Form(request.POST, instance=obj)`**
18. 18. Faqat tizimga kirganlar uchun ochiq view (funksiya)? — **D) `@login_required`**
19. 19. REST API'da resursni o'chirish metodi va javob kodi? — **C) `DELETE, 204`**
20. 20. Model ↔ JSON aylantiruvchi DRF klassi? — **A) `ModelSerializer`**
21. 21. Bir nechta obyektni serializatsiya qilish? — **B) `Serializer(qs, many=True)`**
22. 22. To'liq CRUD API'ni bitta klassda beruvchi DRF klassi va URL'larni avtomatik yaratuvchi vosita? — **D) `ModelViewSet + DefaultRouter`**
23. 23. Boshqa domendagi JS sayt API'ga murojaat qila olishi uchun Django'da nima sozlanadi? — **C) `CORS (django-cors-headers)`**
24. 24. Production'da Django ilovasini ishga tushiruvchi server dasturi? — **A) `gunicorn`**
25. 25. Internetdagi saytda qaysi sozlama noto'g'ri? — **B) `DEBUG = True`**